



Today 11:21 AM

I am working on Lab 4 Vibe Coding. This is the existing game.py from my assigned Zombie Escape repository.

Implement ONLY Task 1 from the README: Health System.

Requirements:

- Give the player 3 HP.
- Each zombie touch reduces HP by 1.
- Add a brief invincibility period after the player is hit so repeated collision does not instantly remove all HP.
- Display the current HP in the HUD.
- The game should end only when HP reaches 0.
- Preserve the existing game structure and mechanics.
- Do not rewrite the entire project.
- Do not add unnecessary external dependencies.
- Explain exactly what you changed and where.
- After giving the changes, tell me how to test every requirement.

https://github.com/arpita764/25_zombie_escape

```
import pygame
import random
import math
import time
```

```
WIDTH, HEIGHT = 800, 560
FPS = 60
BG = (30,35,25)
```

```
class Zombie:
```

```
    SPEED = 1.5
```

```
    def __init__(self, x, y):
        self.rect = pygame.Rect(x, y, 30, 30)
        self.color = (60,140,60)
```

```
self.hp = 3
self.wobble = random.uniform(0, 6.28)
self.frame = 0

def update(self, player_pos):
    px, py = player_pos
    cx, cy = self.rect.center
    dx, dy = px-cx, py-cy
    dist = (dx**2+dy**2)**0.5
    if dist:
        self.rect.x += int(dx/dist*self.SPEED)
        self.rect.y += int(dy/dist*self.SPEED)
    self.frame += 1

def hit(self):
    self.hp -= 1
    return self.hp <= 0

def draw(self, screen):
    wobble_y = int(math.sin(self.frame*0.2)*3)
    draw_rect = self.rect.move(0, wobble_y)
    pygame.draw.rect(screen, self.color, draw_rect,
border_radius=5)
    for ex in [draw_rect.x+6, draw_rect.x+18]:
        pygame.draw.circle(screen, (200,40,40), (ex,
draw_rect.y+10), 4)

def spawn_zombie(width, height, player_rect, margin=120):
    while True:
        x = random.randint(0, width-30)
        y = random.randint(0, height-30)
        rect = pygame.Rect(x, y, 30, 30)
        if not rect.colliderect(player_rect.inflate(margin,
margin)):
            return Zombie(x, y)

SPEED = 4

class Player:
    def __init__(self, x, y):
```

```
self.rect = pygame.Rect(x, y, 32, 32)
self.color = (60,160,220)
self.bullets = []
self.shoot_cooldown = 0

def move(self, keys, width, height):
    dx = dy = 0
    if keys[pygame.K_w] or keys[pygame.K_UP]: dy = -
SPEED
    if keys[pygame.K_s] or keys[pygame.K_DOWN]: dy =
SPEED
    if keys[pygame.K_a] or keys[pygame.K_LEFT]: dx = -
SPEED
    if keys[pygame.K_d] or keys[pygame.K_RIGHT]: dx =
SPEED
    self.rect.x = max(0, min(width-self.rect.width,
self.rect.x+dx))
    self.rect.y = max(0, min(height-self.rect.height,
self.rect.y+dy))
    if self.shoot_cooldown > 0:
        self.shoot_cooldown -= 1

def shoot(self, target_pos):
    if self.shoot_cooldown > 0: return
    cx, cy = self.rect.center
    tx, ty = target_pos
    dx, dy = tx-cx, ty-cy
    dist = (dx**2+dy**2)**0.5
    if dist == 0: return
    vx, vy = dx/dist*10, dy/dist*10
    self.bullets.append(pygame.Rect(cx-4, cy-4, 8, 8))
    self.bullets.append([cx-4, cy-4, vx, vy])
    self.bullets.pop(-2)
    self.shoot_cooldown = 15

def update_bullets(self, width, height):
    live = []
    for b in self.bullets:
        b[0] += b[2]; b[1] += b[3]
        if 0 <= b[0] <= width and 0 <= b[1] <= height:
            live.append(b)
    self.bullets = live
```

```
def draw(self, screen):
    pygame.draw.rect(screen, self.color, self.rect,
border_radius=6)
    for b in self.bullets:
        pygame.draw.circle(screen, (255,220,60), (int(b[0]),
int(b[1])), 5)

class GameEngine:
    def __init__(self):
        pygame.init()
        self.screen = pygame.display.set_mode((WIDTH,
HEIGHT))
        pygame.display.set_caption("Zombie Escape")
        self.clock = pygame.time.Clock()
        self.font = pygame.font.SysFont("monospace", 24)
        self.big_font = pygame.font.SysFont("monospace", 44,
bold=True)
        self.reset()

    def reset(self):
        self.player = Player(WIDTH//2, HEIGHT//2)
        self.zombies = [spawn_zombie(WIDTH, HEIGHT,
self.player.rect) for _ in range(4)]
        self.score = 0
        self.wave = 1
        self.kills = 0
        self.kills_to_next = 8
        self.game_over = False
        self.start_time = time.time()

    def handle_events(self):
        for event in pygame.event.get():
            if event.type == pygame.QUIT: return False
            if event.type == pygame.KEYDOWN and event.key
== pygame.K_r: self.reset()
            if event.type == pygame.MOUSEBUTTONDOWN
and not self.game_over:
                self.player.shoot(event.pos)
        return True

    def update(self):
        if self.game_over: return
```

```
keys = pygame.key.get_pressed()
self.player.move(keys, WIDTH, HEIGHT)
self.player.update_bullets(WIDTH, HEIGHT)
self.score = int(time.time() - self.start_time)

for z in self.zombies:
    z.update(self.player.rect.center)
    if z.rect.colliderect(self.player.rect):
        self.game_over = True

dead = []
for z in self.zombies:
    for b in self.player.bullets[:]:
        bx, by = int(b[0]), int(b[1])
        if z.rect.collidepoint(bx, by):
            if z.hit():
                dead.append(z)
            if b in self.player.bullets:
                self.player.bullets.remove(b)
for z in dead:
    if z in self.zombies:
        self.zombies.remove(z)
        self.kills += 1
        self.score += 10

if self.kills >= self.kills_to_next:
    self.kills = 0
    self.wave += 1
    self.kills_to_next = 8 + self.wave * 2
    for _ in range(self.wave + 3):
        self.zombies.append(spawn_zombie(WIDTH,
HEIGHT, self.player.rect))

def draw(self):
    self.screen.fill(BG)
    for x in range(0, WIDTH, 60):
        pygame.draw.line(self.screen, (40,45,35), (x,0),
(x,HEIGHT), 1)
    for y in range(0, HEIGHT, 60):
        pygame.draw.line(self.screen, (40,45,35), (0,y),
(WIDTH,y), 1)
    for z in self.zombies: z.draw(self.screen)
    self.player.draw(self.screen)
```

```

hud_bg = pygame.Rect(0, 0, WIDTH, 40)
pygame.draw.rect(self.screen, (15,20,15), hud_bg)
hud = self.font.render(
    f"Wave: {self.wave} Score: {self.score} Kills:
{self.kills}/{self.kills_to_next} | WASD Move, Click Shoot, R
Restart",
    True, (160,220,120))
self.screen.blit(hud, (8, 8))
if self.game_over:
    ov = pygame.Surface((WIDTH, HEIGHT),
pygame.SRCALPHA)
    ov.fill((0,0,0,160))
    self.screen.blit(ov, (0,0))
    m = self.big_font.render("DEVoured!", True,
(180,40,40))
    s = self.font.render(f"Wave {self.wave} | Score
{self.score} | Press R", True, (200,200,200))
    self.screen.blit(m, (WIDTH//2-m.get_width())//2,
HEIGHT//2-40))
    self.screen.blit(s, (WIDTH//2-s.get_width())//2,
HEIGHT//2+20))
    pygame.display.flip()

def run(self):
    running = True
    while running:
        running = self.handle_events()
        self.update()
        self.draw()
        self.clock.tick(FPS)
    pygame.quit()

if __name__ == "__main__":
    engine = GameEngine()
    engine.run()
this is my game.py

```


**Pasted code.py**

Python

Continue from the current working Zombie Escape project.
Keep Tasks 1, 2 and 3 working.

Implement ONLY Task 4 from the README: Zombie Types.

Add:

1. Fast Zombie:

- higher movement speed than the standard zombie
- 1 HP
- smaller size

2. Tank Zombie:

- lower movement speed than the standard zombie
- 6 HP
- larger size

Requirements:

- Both zombie types must spawn during waves alongside standard zombies.
- Their different speed, size and HP should be observable during gameplay.
- Wave progression must continue working.
- Preserve the health system, ammo/reload system, barrels and existing mechanics.
- Do not rewrite the entire project.
- Do not add unnecessary dependencies.

- Explain exactly what changed and provide a testing checklist.

this is the game.py including task 1,2 and 3

Yes — I see the likely problem.

Your `initial reset()` still creates only standard zombies:

Python

Run

```
self.zombies = [spawn_zombie(WIDTH, HEIGHT, self.player.rect) for _ in range(4)]
```

So when you start the game, you will **not see a Fast Zombie in Wave 1**. The Fast/Tank zombies are only created when the wave advances. That makes Test 2 confusing.

Let's make Task 4 easier to verify while keeping the existing mechanics.

Fix: spawn all 3 types from the beginning

1. Replace this line in `reset()`

Current:

Python

Run

```
self.zombies = [spawn_zombie(WIDTH, HEIGHT, self.player.rect) for _ in range(4)]
```

Replace it with:

Python

Run

```
self.zombies = [  
    spawn_zombie(WIDTH, HEIGHT, self.player.rect, Zombie)  
    for _ in range(4)  
]  
  
self.zombies.append(  
    spawn_zombie(WIDTH, HEIGHT, self.player.rect, FastZombie)
```

```

    )

    self.zombies.append(
        spawn_zombie(WIDTH, HEIGHT, self.player.rect, TankZombie)
    )

```

Now the game starts with:

- 4 Standard Zombies
- 1 Fast Zombie
- 1 Tank Zombie

So you can immediately test Task 4.

2. Make sure your FastZombie class is exactly this

It should be after the Zombie class:

Python

Run

```

class FastZombie(Zombie):
    SPEED = 3.0
    SIZE = 20
    HP = 1
    COLOR = (220,180,40)

```

And Tank:

Python

Run

```

class TankZombie(Zombie):
    SPEED = 0.75
    SIZE = 42
    HP = 6
    COLOR = (120,70,160)

```

3. Make sure spawn_zombie() is this

Python

Run

```

def spawn_zombie(width, height, player_rect, zombie_class=Zombie, margin=120):
    size = zombie_class.SIZE

    while True:
        x = random.randint(0, width-size)
        y = random.randint(0, height-size)

```

```
rect = pygame.Rect(x, y, size, size)

if not rect.colliderect(player_rect.inflate(margin, margin)):
    return zombie_class(x, y)
```

The important part is:

Python

Run

```
return zombie_class(x, y)
```

If you still have:

Python

Run

```
return Zombie(x, y)
```

then every zombie will become a standard zombie, even if you call:

Python

Run

```
spawn_zombie(..., FastZombie)
```

This is the most important thing to check.

4. Test it now

Run:

Bash

```
python game.py
```

You should now immediately see:

Green medium zombie

```
Standard
30 x 30
3 HP
Speed 1.5
```

Yellow small zombie

FAST
20 × 20
1 HP
Speed 3.0

Purple large zombie

TANK
42 × 42
6 HP
Speed 0.75

The colors are intentional so you can easily distinguish them.

5. Test Fast Zombie specifically

Find the small yellow zombie.

Shoot it **once**.

It should immediately disappear.

If it takes multiple shots, check that you have:

Python

Run

```
class FastZombie(Zombie):  
    SPEED = 3.0  
    SIZE = 20  
    HP = 1  
    COLOR = (220,180,40)
```

6. Test its speed

Let a standard zombie and the Fast Zombie chase you.

You should notice:

Fast Zombie	→ much faster
Standard Zombie	→ normal

because:

Python

Run

```
FastZombie.SPEED = 3.0
Zombie.SPEED = 1.5
```

7. If the Fast Zombie still doesn't appear

Add this temporary line inside `reset()` after creating the zombies:

Python

Run

```
print([(type(z).__name__, z.hp, z.rect.size) for z in self.zombies])
```

Run the game from the terminal.

You should see something similar to:

```
[('Zombie', 3, (30, 30)),
 ('Zombie', 3, (30, 30)),
 ('Zombie', 3, (30, 30)),
 ('Zombie', 3, (30, 30)),
 ('FastZombie', 1, (20, 20)),
 ('TankZombie', 6, (42, 42))]
```

If you see that, the Task 4 spawning is working.

You can remove the `print()` afterward.

Most likely issue

If your screen shows only the original green zombies, **check** `spawn_zombie()` **first**. If it still says:

Python

Run

```
return Zombie(x, y)
```

that is why the Fast and Tank types aren't appearing. Your uploaded original had exactly that standard-only return before the Task 4 changes.

Pasted code

Sources



test 5 not working in first wave, after all the zombies are killed nothing happens

You're right. The issue is in the **wave progression condition**, not the Fast/Tank spawning.

Your original code uses:

Python

Run

```
if self.kills >= self.kills_to_next:
```

and starts with:

Python

Run

```
self.kills_to_next = 8
```

But after our Task 4 change, you started with **6 zombies**:

- 4 Standard
- 1 Fast
- 1 Tank

So after killing all 6:

```
kills = 6  
kills_to_next = 8
```

Therefore:

Python

Run

```
6 >= 8
```

is false, so Wave 2 never starts.

Fix

The cleanest solution is to keep your original wave progression based on **8 kills**, while making the first wave contain enough zombies to reach that threshold.

Replace your `reset()` zombie creation

Currently you probably have:

Python

Run

```

self.zombies = [
    spawn_zombie(WIDTH, HEIGHT, self.player.rect, Zombie)
    for _ in range(4)
]

self.zombies.append(
    spawn_zombie(WIDTH, HEIGHT, self.player.rect, FastZombie)
)

self.zombies.append(
    spawn_zombie(WIDTH, HEIGHT, self.player.rect, TankZombie)
)

```

Replace it with:

Python

Run

```

self.zombies = [
    spawn_zombie(WIDTH, HEIGHT, self.player.rect, Zombie)
    for _ in range(6)
]

self.zombies.append(
    spawn_zombie(WIDTH, HEIGHT, self.player.rect, FastZombie)
)

self.zombies.append(
    spawn_zombie(WIDTH, HEIGHT, self.player.rect, TankZombie)
)

```

Now the first wave has:

```

6 Standard
1 Fast
1 Tank
-----
8 total

```

So after killing all 8:

```

kills = 8
kills_to_next = 8

```

and this existing code:

Python

Run

```

if self.kills >= self.kills_to_next:
    self.kills = 0
    self.wave += 1
    self.kills_to_next = 8 + self.wave * 2

    for _ in range(self.wave + 1):
        self.zombies.append(
            spawn_zombie(WIDTH, HEIGHT, self.player.rect, Zombie)
        )

        self.zombies.append(
            spawn_zombie(WIDTH, HEIGHT, self.player.rect, FastZombie)
        )

        self.zombies.append(
            spawn_zombie(WIDTH, HEIGHT, self.player.rect, TankZombie)
        )

```

will trigger.

What you'll get

Wave 1:

```

6 Standard
1 Fast
1 Tank
= 8 zombies

```

Kill all 8 → **Wave 2**

Wave 2:

range(self.wave + 1) means range(3) :

```

3 Standard
3 Fast
3 Tank
= 9 zombies

```

Kill 10 or more according to the updated kills_to_next → **Wave 3**.

One more thing to check

Make sure your wave-spawning section actually contains the three types:

Python

Run

```

for _ in range(self.wave + 1):
    self.zombies.append(
        spawn_zombie(WIDTH, HEIGHT, self.player.rect, Zombie)
    )

    self.zombies.append(
        spawn_zombie(WIDTH, HEIGHT, self.player.rect, FastZombie)
    )

    self.zombies.append(
        spawn_zombie(WIDTH, HEIGHT, self.player.rect, TankZombie)
    )

```

Don't change your `kills_to_next` logic. The problem was simply that the first wave had 6 zombies but required 8 kills.

Test again

1. Start the game.
2. You should see **8 zombies total**.
3. Kill all 8.
4. HUD should change from:

Wave: 1

to:

Wave: 2

5. New Standard, Fast, and Tank zombies should appear.

This preserves the existing wave progression and only adjusts the initial zombie count to match the existing `kills_to_next = 8` requirement.



Pasted code(1).py
Python

Explosive Barrel

Place 4 barrels on the map. A bullet hitting a barrel triggers an explosion that destroys all zombies within a

fixed radius. Remove the barrel after it explodes.

Done when: barrels are visible on the map, a bullet collision triggers the explosion effect, nearby zombies are removed, and the barrel disappears.

this is game.py including task1,2,3 and 4.inlcude the explosion effect

Yes. Your current Task 3 already **destroys nearby zombies and removes the barrel**, but there is no visible explosion animation. The relevant collision code is at lines 264–296, while barrels are currently drawn normally at lines 336–337. Pasted code +1

We can add a **simple expanding explosion effect** without changing Tasks 1, 2, or 4.

1. Add an Explosion class

Put this **after the** `Barrel` class and before `spawn_zombie()`.

Python

Run

```
class Explosion:
    def __init__(self, x, y, radius=90):
        self.x = x
        self.y = y
        self.radius = radius
        self.max_radius = radius
        self.duration = 0.35
        self.start_time = time.time()

    def update(self):
        elapsed = time.time() - self.start_time
        return elapsed < self.duration

    def draw(self, screen):
        elapsed = time.time() - self.start_time
        progress = min(elapsed / self.duration, 1.0)

        current_radius = int(self.max_radius * progress)

        # Outer explosion ring
        pygame.draw.circle(
            screen,
            (255, 120, 20),
            (self.x, self.y),
            current_radius,
            5
        )
```

```
# Inner explosion
inner_radius = max(5, int(current_radius * 0.55))

pygame.draw.circle(
    screen,
    (255, 210, 60),
    (self.x, self.y),
    inner_radius
)
```

This creates an explosion that:

- starts small,
- expands outward,
- reaches the existing 90-pixel explosion radius,
- lasts about **0.35 seconds**.

No external dependency is required.

2. Create an explosion list

In your `reset()` method, you currently initialize the barrels around lines 222–226.

Pasted code

Immediately **after** the barrel creation, add:

Python

Run

```
self.explosions = []
```

So this section becomes:

Python

Run

```
self.barrels = []

for _ in range(4):
    self.barrels.append(
        spawn_barrel(
            WIDTH,
            HEIGHT,
            self.player.rect,
            self.barrels
        )
    )
```

```
self.explosions = []
```

3. Add the explosion when the barrel is hit

Your current code has:

Python

Run

```
if barrel.rect.collidepoint(bx, by):
    exploded_barrels.append(barrel)

    if b in self.player.bullets:
        self.player.bullets.remove(b)
```

This is around lines 268–276.

Pasted code

Change it to:

Python

Run

```
if barrel.rect.collidepoint(bx, by):
    exploded_barrels.append(barrel)

    # Create visual explosion
    barrel_x, barrel_y = barrel.rect.center
    self.explosions.append(
        Explosion(barrel_x, barrel_y, explosion_radius)
    )

    if b in self.player.bullets:
        self.player.bullets.remove(b)

    for z in self.zombies[:]:
        zombie_x, zombie_y = z.rect.center
        distance = math.sqrt(
            (zombie_x - barrel_x) ** 2 +
            (zombie_y - barrel_y) ** 2
        )

        if distance <= explosion_radius:
            self.zombies.remove(z)
            self.kills += 1
            self.score += 10

    break
```

Important

You should remove the old duplicate lines:

Python

Run

```
barrel_x, barrel_y = barrel.rect.center

for z in self.zombies[:]:
```

because they are now included in the updated block above.

So your entire barrel collision section should look like this:

Python

Run

```
# Check bullet-barrel collisions
exploded_barrels = []
explosion_radius = 90

for barrel in self.barrels:
    for b in self.player.bullets[:]:
        bx, by = int(b[0]), int(b[1])

        if barrel.rect.collidepoint(bx, by):
            exploded_barrels.append(barrel)

            # Create visual explosion
            barrel_x, barrel_y = barrel.rect.center
            self.explosions.append(
                Explosion(barrel_x, barrel_y, explosion_radius)
            )

            if b in self.player.bullets:
                self.player.bullets.remove(b)

            for z in self.zombies[:]:
                zombie_x, zombie_y = z.rect.center

                distance = math.sqrt(
                    (zombie_x - barrel_x) ** 2 +
                    (zombie_y - barrel_y) ** 2
                )

                if distance <= explosion_radius:
                    self.zombies.remove(z)
                    self.kills += 1
                    self.score += 10

            break
```

```
for barrel in exploded_barrels:
    if barrel in self.barrels:
        self.barrels.remove(barrel)
```

4. Update the explosions

After the barrel collision code, add:

Python

Run

```
# Update active explosions
self.explosions = [
    explosion
    for explosion in self.explosions
    if explosion.update()
]
```

So immediately after:

Python

Run

```
for barrel in exploded_barrels:
    if barrel in self.barrels:
        self.barrels.remove(barrel)
```

add:

Python

Run

```
# Update active explosions
self.explosions = [
    explosion
    for explosion in self.explosions
    if explosion.update()
]
```

The explosion stays visible for its short animation duration and then disappears.

5. Draw the explosion

In your `draw()` method, you currently have:

Python

Run

```
for barrel in self.barrels:
    barrel.draw(self.screen)

for z in self.zombies:
    z.draw(self.screen)
```

Your barrels are currently drawn at this point.

Pasted code

Change this to:

Python

Run

```
for barrel in self.barrels:
    barrel.draw(self.screen)

for explosion in self.explosions:
    explosion.draw(self.screen)

for z in self.zombies:
    z.draw(self.screen)
```

So the full relevant section is:

Python

Run

```
for barrel in self.barrels:
    barrel.draw(self.screen)

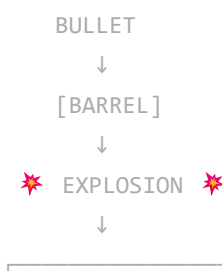
for explosion in self.explosions:
    explosion.draw(self.screen)

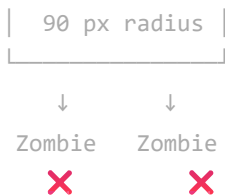
for z in self.zombies:
    z.draw(self.screen)

self.player.draw(self.screen)
```

What you now have

When you shoot a barrel:





The sequence is:

1. Bullet hits barrel.
2. Bullet disappears.
3. Explosion animation starts.
4. Explosion expands to **90 pixels**.
5. Zombies within 90 pixels are immediately removed.
6. Kill count and score are updated.
7. Barrel disappears.
8. Explosion animation disappears after ~0.35 seconds.

The existing **90-pixel radius** is preserved, so this is only adding the requested visual effect to your existing Task 3 behavior. Your Task 4 zombie classes continue to work because the explosion operates on the same `self.zombies` list.

Pasted code

Testing checklist

Test 1 — Four barrels

Start the game.

You should see exactly:

```
4 barrels
```

Test 2 — Barrel is visible

Verify each barrel is clearly visible before being shot.

Test 3 — Shoot barrel

Aim at a barrel and click.

You should see:

```
Bullet → Barrel → EXPANDING EXPLOSION
```

The explosion should be visible for a fraction of a second.

Test 4 — Barrel disappears

After the explosion, the barrel should be gone.

For example:

4 → 3 → 2 → 1 → 0

as you destroy them.

Test 5 — Nearby zombies die

Place/allow zombies near a barrel.

Shoot the barrel.

All zombies within the fixed **90-pixel radius** should disappear.

Test 6 — Distant zombie survives

Shoot a barrel while another zombie is more than 90 pixels away.

The distant zombie should remain.

Test 7 — Fast Zombie

Put the small Fast Zombie near a barrel and explode it.

It should be destroyed normally.

Test 8 — Tank Zombie

Put the large Tank Zombie near a barrel and explode it.

It should also be destroyed by the explosion regardless of its 6 HP.

Test 9 — Existing systems

Verify that:

- **HP** still decreases on zombie contact.
- **1-second invincibility** still works.
- **Ammo** still goes from 12 → 0.
- **Reload** still takes 2 seconds.
- Direct bullets still damage zombies.
- Wave progression still works.
- Fast and Tank zombies still spawn.

So you are **not changing Tasks 1, 2, or 4**—you're only adding the missing visual explosion effect to Task 3.

Sources



Is this conversation helpful so far?



ChatGPT can make mistakes. Check important info.